USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Milestone 3 Verification Report

`UM-NATOS-010`

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-010	1.0 · 2026-08-14	**PASS** — all three exit criteria met on hardware	Internal

1. Abstract

Milestone 3 delivers the kernel allocator and the arena model that bytecode applications will run inside. Its exit criteria (UM-NATOS-007 §5) are narrow and measurable: no leak across 10,000 allocate/free cycles, arena bounds queryable by the interpreter, and exhaustion that fails rather than corrupts.

All three pass. The more consequential outcome is not the pass — it is the number the allocator finally puts on the DRAM budget, which settles a question M5 could otherwise only have guessed at. See §7.

2. Design decisions

2.1 Address-ordered list with physical links

The heap is a doubly-linked list of blocks in address order, covering the region with no gaps. Each block carries a 16-byte header: both physical neighbours, its payload size, and a magic word that doubles as the used/free flag.

	Segregated free lists	Address-ordered physical list (selected)
Allocation	O(1) typical	O(n) first fit
Coalescing	Needs a search or boundary tags	O(1) both directions
State to verify	Several lists plus size classes	One list, one invariant
Overhead	Lower	16 B per block

Throughput is not the constraint. This allocator serves arena creation and a small number of kernel structures, not millions of short-lived objects. What matters is that the structure can be *checked*, and a single address-ordered list tiles the heap exactly, which makes the invariant trivially statable: every block's payload must end precisely where the next block's header begins.

2.2 The magic word earns its four bytes

`BLK_FREE` / `BLK_USED` are implausible-as-data constants that also encode state, so a header is either valid or obviously not. A double free, an interior pointer, or a wild pointer is therefore **counted and refused** rather than linked into the list.

On a kernel with no memory protection (UM-NATOS-001 §4.2) this is not defensiveness for its own sake. A corrupted free list surfaces as a fault in unrelated code much later, and M2 already demonstrated how expensive it is to debug a symptom that appears far from its cause.

2.3 Arenas are not resizable

An arena's base is the value the interpreter holds and bounds-checks against. Allowing it to move would mean every bytecode program must survive its memory being relocated underneath it — a much harder property to guarantee than telling a program its size once, at start.

2.4 The heap is placed by the linker, not sized by hand

`_heap_start` follows `.bss`; `_heap_end` is the top of DRAM less a 4 KB boot stack reservation. A hand-chosen base would silently shrink the heap whenever a static array grew, and surface much later as an allocation failure with no obvious connection to the change that caused it.

The boot stack reservation is kept permanently even though the boot context is abandoned at handoff, because the heap must not be adjacent to a stack that grew further than expected before that point. It costs 2% of DRAM.

3. Implementation

`heap_alloc` is first fit. A block is split only when the remainder can hold a header plus a useful payload; below that the leftover stays with the allocation. Splitting a block consumes payload to create a header, and coalescing returns it — so the *usable* total moves as the heap fragments, and `heap_total()` tracks that honestly rather than reporting a fixed figure.

`heap_free` validates the magic word, clears it, then merges forward and backward. Merging is unconditional in both directions, which is what makes the "two adjacent free blocks" invariant checkable.

`heap_check()` walks the list and returns a non-zero code identifying the first violated invariant — corrupt header, broken back link, misalignment, a gap or overlap between blocks, a missed coalesce, or accounting that disagrees with the walk. It is called after every phase of the self-test rather than only at the end, so a failure names the operation that caused it.

4. Verification method

The self-test runs single-threaded in `kmain` before the tick is armed. Nothing in it can be disturbed by a context switch, so an M3 failure cannot be blamed on M2 — and vice versa.

Criterion 1 allocates and frees 10,000 times across 8 rotating slots with pseudo-random sizes from 16 to 515 bytes, so blocks are split and coalesced continuously. The check is not merely that free bytes return to baseline: the **largest free block** and the **block count** must also return, because a heap that has fragmented into unusable slivers still reports the correct number of free bytes. That is the failure a naive leak test misses.

Criterion 2 creates two arenas and exercises ten bounds cases, including the ones a naive check gets wrong.

Criterion 3 requests more than the heap holds, then attempts a double free and a free of a stack address.

5. Results

```
heap         : 166432 B usable, largest 166432 B, blocks 1

[1] no leak  : PASS  after 10000 cycles  free=166432 largest=166432
               blocks=1 check=0
[3] oom safe : PASS  oversize=NULL fails=1 bad_frees=2 check=0
[2] arenas   : PASS  live=2 committed=5120 B  base=0x3ffb27e0 len=4096
arenas freed : check=0 free=166432/166432 rejects=1 high_water=5120 B
```

5.1 Criterion 1 — no leak

Free bytes, largest free block, and block count all return **exactly** to their pre-test values, and the heap collapses back to a single block. Structural check clean. No allocation failed during the run.

Returning to one block is the strong form of the result: it says every one of the ~10,000 splits was undone by a matching coalesce, not merely that the byte totals happen to balance.

5.2 Criterion 2 — arena bounds

All ten cases behaved correctly: exact fit, last byte, zero-length access, one past the end, one before the base, one byte too long, a length chosen to wrap the address space, an address belonging to a different arena, and a bogus id.

The wrap case is why `arena_contains()` works in the offset domain. Testing `addr + len` directly overflows and reports success for an access far outside the arena — precisely what a hostile length would target. Since this function is the only thing standing between a bytecode program and the rest of DRAM, it is implemented once and exported rather than reimplemented per caller.

Arena memory is zeroed at creation and verified zero, so an application cannot read what a previous occupant left.

5.3 Criterion 3 — exhaustion and refusal

An oversized request returned `NULL` and incremented the failure counter, leaving the heap byte-identical and structurally clean. The double free and the stack-address free were both counted (`bad_frees=2`) and neither perturbed the list. `rejects=1` records destroying an already-destroyed arena.

5.4 M2 regression

The M2 workload continued under the same image: 3,418 ticks, switches 1140/1139/1139, guards intact, `corrupt=0` . Task stack headroom improved from 463 to 480 words with switch tracing compiled out.

6. Metrics

Quantity	Value
Image size	6,720 B
Heap, usable	166,432 B
Header overhead	16 B per block
Boot stack reserved	4,096 B
Alloc/free cycles	10,000
Blocks after test	1 (baseline)
High-water mark	5,120 B
Allocation failures	1 (the deliberate one)
Refused frees	2 (both deliberate)
`heap_check()` failures	0

7. The DRAM budget, now measured

UM-NATOS-007 §5 named arena sizing as M3's principal risk and asked for measurements rather than guesses. Here they are.

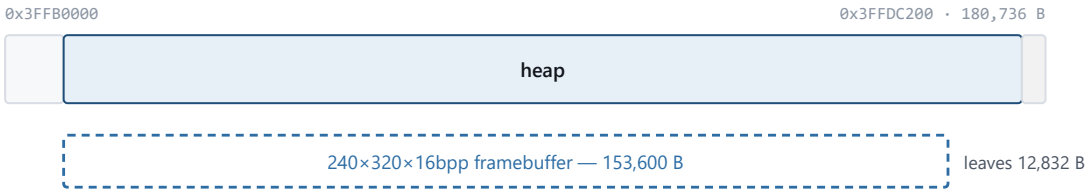

Figure 1. *Measured DRAM split after M3. A full 16-bit framebuffer would take 92% of the heap, which is the constraint M5 has to design around.*

Of 180,736 B of DRAM: 10,160 B is kernel data, `.rodata` and task stacks; 4,096 B is the boot stack reservation; **166,432 B is allocatable.**

The consequence is sharper than expected:

Consumer	Bytes	Share of heap
Full 240×320 16-bit framebuffer	153,600	92.3%
Remaining for all arenas	12,832	7.7%

A full-screen 16-bit framebuffer and any meaningful set of concurrent VM applications cannot coexist in internal DRAM. With one committed, 12.8 KB is left — a single small arena, with no room for a second application.

§7.2 argues the framebuffer should not exist at all, which dissolves this conflict rather than resolving it. The table is retained because the figure is what rules the naive approach out, and because it is the number to check against if a future design reintroduces a local copy for compositing.

This is a genuine constraint, not a tuning problem, and it lands on M5 (display) rather than here. The options, none yet chosen:

1. **No full framebuffer.** Drive the ILI9341 from a line or tile buffer and push updates incrementally. A 240-pixel line buffer is 480 B. Costs redraw complexity, keeps essentially the whole heap for applications.
2. **Reduced colour depth.** 8bpp halves it to 76,800 B — still 46% of the heap.
3. **Partial/dirty-region updates.** A buffer covering only the changed region.
4. **External PSRAM. Confirmed absent.** The chip reports as ESP32-D0WD-V3 rev 3.1 with features `WiFi, BT, Dual Core, 240MHz, VRef calibration in efuse, Coding Scheme None` — no embedded PSRAM, and none on the module. This option is closed without a hardware change.

Option 1 is the only one that preserves the arena budget outright, and it should be treated as the default until measurement says otherwise.

7.1 Why flash cannot hold the framebuffer

The obvious question, given 4 MB of flash against 176 KB of DRAM. The answer is no, and for reasons of mechanism rather than capacity:

- Flash is memory-mapped **read-only** at runtime. It can be read through the cache; it cannot be the target of a store instruction.
- Writing requires erasing a 4 KB sector and then programming pages. The erase is mandatory — programming can only clear bits, not set them — and costs tens of milliseconds, roughly three orders of magnitude more than a frame's budget.
- Endurance is ~100,000 erase cycles per sector. A 153,600 B framebuffer spans about 38 sectors; at 30 fps each is erased 30 times per second, exhausting rated endurance in **under an hour**. At 1 fps it is about a day.
- Flash writes require the cache to be disabled, stalling any code executing from flash.

Flash as framebuffer is therefore not slow-but-workable. It destroys the part.

7.2 The framebuffer is not needed in the first place

The premise deserves challenging rather than optimising. **The ILI9341 contains its own frame memory** — a GRAM of 240×320 at 18bpp, roughly 172,800 bytes — and drives the panel from it autonomously. Pixels written to the controller stay there; the host is not refreshing a surface continuously.

The ESP32 therefore never requires a full local copy. It requires a *transfer* buffer: 480 B for one 240-pixel line at 16bpp, 2,048 B for a 32×32 tile. The 153,600 B in §7 was never a hardware requirement — it is the cost of keeping a redundant second copy of something the display already stores.

The one case that genuinely wants a local copy is read-modify-write rendering — alpha blending, scrolling, compositing — where the previous pixel value is an input. The ILI9341 does support pixel read-back over SPI, but it is slow and unreliable on boards of this class, so a dirty-region buffer sized to the area actually being composited is the appropriate middle ground.

This strengthens option 1 from "the least bad choice" to "the correct one".

7.3 What flash is genuinely worth using for

Flash does not solve the framebuffer question, but it addresses what actually consumes DRAM today and will consume more later:

Candidate	Present cost	After moving to flash
`.rodata`	DRAM (see §2.4 and `linker.ld`)	0 B DRAM, read via cache
Fonts, sprites, UI bitmaps	Would be DRAM	0 B DRAM
Application bytecode	Would occupy an arena	Arena holds only app *data*

`.rodata` sits in DRAM today because IRAM cannot serve unaligned reads (UM-NATOS-004). Memory-mapped flash has no such restriction, so the constraint that forced the current placement does not apply. Realising this requires enabling the flash cache — the work deferred in UM-NATOS-004 §3 — and is therefore a prerequisite worth scheduling before the asset-heavy milestones rather than during them.

> **DONE — see UM-NATOS-011.** The `.rodata` row of this table is now fact rather than proposal: it is mapped from flash, and the heap figure quoted throughout this report has risen from 166,432 B to **167,680 B** as a result. The other two rows remain proposals, but the mechanism they depend on exists.

Budget context: 4 MB of flash against a 6,720 B kernel image. Availability is not the constraint; cache configuration is.

8. What M3 does not establish

- **No allocator concurrency.** `heap_alloc` / `heap_free` are task-context only. There is no lock, because the kernel has no locking primitive yet. An allocation from an interrupt handler would corrupt the list, and nothing currently prevents one.
- **No arena enforcement.** M3 provides `arena_contains()`; nothing *calls* it yet. The isolation guarantee arrives with the interpreter in M4, and until then arenas are bookkeeping, not protection.
- **No fragmentation characterisation under realistic load.** The test's size distribution is uniform and its lifetimes are uniform. Real workloads are neither, and a first-fit allocator's worst case is workload-shaped.
- **No allocation latency measurement.** First fit is O(n) in block count, which is fine at today's block counts and was not measured.
- **Arena count is fixed** at `ARENA_MAX = 4`, statically.

9. References

- UM-NATOS-001 §4.2 — isolation model; why bounds checking is the whole guarantee
- UM-NATOS-004 §4 — DRAM regions and the framebuffer estimate this report refines
- UM-NATOS-007 §5 — M3 deliverable, principal risk, and exit criteria
- UM-NATOS-009 — M2, whose failure mode motivated §2.2
- `kernel/heap.c` — allocator and `heap_check()` invariants

- `kernel/arena.c` — arena lifecycle and `arena_contains()`
- `kernel/linker.ld` — `_heap_start` / `_heap_end` placement